

Stack, Queue & Monotonic Deque

1. Stack & Queue cơ bản

- Stack (LIFO - Last In First Out): push, pop, top đều $O(1)$.
- Queue (FIFO - First In First Out): push, pop, front đều $O(1)$.
- Deque (Double-Ended Queue): thêm/xóa cả hai đầu trong $O(1)$.

C++

```
#include <stack>
#include <queue>
#include <deque>

stack<int> st;
st.push(5); st.top(); st.pop();

queue<int> q;
q.push(5); q.front(); q.pop();

deque<int> dq;
dq.push_back(5); dq.push_front(3);
dq.pop_back(); dq.pop_front();
```

Python

```
from collections import deque

# Stack: dùng list, append / pop o cuối deu  $O(1)$ 
st = []
st.append(5); st[-1]; st.pop()

# Queue: dùng deque, KHÔNG dùng list (list.pop(0) la  $O(n)!$ )
q = deque()
q.append(5); q[0]; q.popleft()

# Deque: them/xoa ca hai dau trong  $O(1)$ 
dq = deque()
dq.append(5); dq.pop(); dq.appendleft(3); dq.popleft()
```

Bây Python: KHÔNG dùng List làm queue. List.pop(0) là $O(n)$ vì phải dịch chuyển toàn bộ phần tử. Luôn dùng collections.deque cho queue và deque.

2. Bài toán mở đầu: Dấu ngoặc hợp lệ

Đề: Cho xâu gồm ()[]{}. Kiểm tra xem có hợp lệ không.

Ý tưởng: Mở thì push, đóng thì pop và so sánh.

C++

```

for (char c : s) {
    if (c == '(' || c == '[' || c == '{') st.push(c);
    else {
        if (st.empty()) return false;
        char x = st.top(); st.pop();
        if ((c == ')' && x != '(') ||
            (c == ']' && x != '[') ||
            (c == '}' && x != '{')) return false;
    }
}
return st.empty();

```

Python

```

pair = {'(': ')', '[': ']', '{': '}'
st = []
for c in s:
    if c in '([{':
        st.append(c)
    else:
        if not st or st.pop() != pair[c]:
            return False
return not st

```

Đây là mẫu kinh điển – nhớ thật kỹ!

3. Monotonic Stack - Stack đơn điệu

Ý tưởng: Duy trì một stack mà các phần tử bên trong luôn tăng (hoặc luôn giảm) từ đáy lên đỉnh.

Bài toán: Nearest Smaller Value (giá trị nhỏ hơn gần nhất bên trái)

Đề: Cho dãy $a_1, a_2, \dots, a_n$. Với mỗi i , tìm j lớn nhất với $j < i$ và $a_j < a_i$. Nếu không có, in 0.

Cách $O(n^2)$: Với mỗi i , duyệt ngược j – quá chậm.

Cách $O(n)$ với monotonic stack: Duy trì stack lưu các index j với a_j tăng dần. Khi xét i mới:

1. Bật (pop) khỏi stack mọi j có $a_j \geq a_i$.
2. Đỉnh stack hiện tại (nếu có) chính là đáp án.
3. Push i vào stack.

Stack luôn giữ giá trị tăng dần (đáy $\rightarrow$ đỉnh). Phần tử mới bật mọi giá trị $\geq$ chính nó.

Hình dung – chạy tay với $a = [3, 1, 4, 1, 5]$:

Stack chứa chỉ số sao cho giá trị tương ứng tăng dần từ đáy $\rightarrow$ đỉnh.

$i = 1, a[1] = 3$:

- stack rỗng $\rightarrow ans[1] = 0$
- push 1
- stack idx: [1], stack val: [3]

$i = 2, a[2] = 1$:

- $a[1] = 3 \geq 1 \rightarrow \text{pop } 1$
- stack rỗng $\rightarrow ans[2] = 0$
- push 2
- stack idx: [2], stack val: [1]

$i = 3, a[3] = 4$:

- $a[2] = 1 < 4 \rightarrow \text{KHÔNG pop}$
- $\text{top} = 2 \rightarrow ans[3] = 2$
- push 3
- stack idx: [2, 3], stack val: [1, 4]

$i = 4, a[4] = 1$:

- $a[3] = 4 \geq 1 \rightarrow \text{pop } 3$
- $a[2] = 1 \geq 1 \rightarrow \text{pop } 2$
- stack rỗng $\rightarrow ans[4] = 0$
- push 4
- stack idx: [4], stack val: [1]

$i = 5, a[5] = 5$:

- $a[4] = 1 < 5 \rightarrow \text{KHÔNG pop}$
- $\text{top} = 4 \rightarrow ans[5] = 4$
- push 5
- stack idx: [4, 5], stack val: [1, 5]

Đáp án: $ans = [0, 0, 2, 0, 4]$. Mỗi chỉ số được push 1 lần, pop tối đa 1 lần $\Rightarrow$ tổng $O(n)$ amortized.

C++

```
stack<int> st;
for (int i = 1; i <= n; i++) {
    while (!st.empty() && a[st.top()] >= a[i]) st.pop();
    ans[i] = st.empty() ? 0 : st.top();
    st.push(i);
}
```

Python

```
ans = [0] * (n + 1)
for i in range(1, n + 1):
    while st and a[st[-1]] >= a[i]:
        st.pop()
    ans[i] = st[-1] if st else 0
    st.append(i)
```

Tại sao $O(n)$? Mỗi phần tử được push 1 lần và pop tối đa 1 lần $\rightarrow$ amortized $O(n)$.

Khi nào dùng monotonic stack?

- Tìm "phần tử gần nhất bên trái/phải nhỏ hơn / lớn hơn"
- Tính diện tích lớn nhất hình chữ nhật trong histogram
- Dạng "với mỗi i , tìm khoảng $[l, r]$ mà a_i là min/max"

4. Monotonic Deque cho cửa sổ trượt

Bài toán: Cho dãy $a_1, \dots, a_n$ và k . Với mỗi cửa sổ độ dài k liên tiếp, in giá trị lớn nhất.

Cách ngây thơ: $O(nk)$ – chậm.

Cách $O(n)$ với deque: Duy trì deque chứa index với giá trị a giảm dần từ đầu đến cuối deque.

Khi trượt sang phải tới i :

1. Xóa từ đuôi các phần tử có a nhỏ hơn a_i (chúng không bao giờ là max trong cửa sổ chứa i).
2. Push i vào đuôi.
3. Xóa từ đầu nếu index $\leq i - k$ (đã ngoài cửa sổ).
4. Đầu deque là index của max.

Bất biến: giá trị trong deque GIẢM DẦN từ đầu xuống đuôi. Đầu deque luôn = max của cửa sổ.

Hình dung – chạy tay với $a = [3, 1, 5, 2, 4]$, $k = 3$:

Deque luôn chứa chỉ số sao cho giá trị tương ứng giảm dần từ đầu $\rightarrow$ cuối.

$i = 1$, $a[1] = 3$:

- Pop đuôi (rỗng): skip
- Push 1 $\rightarrow$ deque (idx): [1] $\rightarrow$ giá trị: [3]

$i = 2$, $a[2] = 1$:

- $a[1] = 3 > a[2] = 1 \rightarrow$ KHÔNG pop
- Push 2 $\rightarrow$ deque (idx): [1, 2] $\rightarrow$ giá trị: [3, 1]

$i = 3, a[3] = 5$:

- $a[2] = 1 \leq 5 \rightarrow \text{pop } 2 \rightarrow [1]$
- $a[1] = 3 \leq 5 \rightarrow \text{pop } 1 \rightarrow []$
- Push 3 $\rightarrow$ deque (idx): [3] $\rightarrow$ giá trị: [5]
- Cửa sổ [1..3]: $\text{max} = a[\text{dq.front}()] = a[3] = 5$

$i = 4, a[4] = 2$:

- $a[3] = 5 > 2 \rightarrow$ KHÔNG pop
- Push 4 $\rightarrow$ deque (idx): [3, 4] $\rightarrow$ giá trị: [5, 2]
- $\text{Front} = 3 \geq i - k + 1 = 2 \rightarrow$ trong cửa sổ
- Cửa sổ [2..4]: $\text{max} = a[3] = 5$

$i = 5, a[5] = 4$:

- $a[4] = 2 \leq 4 \rightarrow \text{pop } 4 \rightarrow [3]$
- $a[3] = 5 > 4 \rightarrow$ KHÔNG pop
- Push 5 $\rightarrow$ deque (idx): [3, 5] $\rightarrow$ giá trị: [5, 4]
- $\text{Front} = 3 < i - k + 1 = 3$? Không ($3 = 3$) $\rightarrow$ vẫn trong cửa sổ
- Cửa sổ [3..5]: $\text{max} = a[3] = 5$

Quan sát: giá trị trong deque luôn giảm dần $\rightarrow \text{front} = \text{max}$. Mỗi chỉ số chỉ bị push 1 lần và pop tối đa 1 lần $\Rightarrow$ tổng $O(n)$.

C++

```
for (int i = 1; i <= n; i++) {
    while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
    dq.push_back(i);
    if (dq.front() <= i - k) dq.pop_front();
    if (i >= k) cout << a[dq.front()] << ' ';
}
```

Python

```
from collections import deque

dq = deque()
out = []
for i in range(n):
    while dq and a[dq[-1]] <= a[i]:
        dq.pop()
    dq.append(i)
    if dq[0] <= i - k:
        dq.popleft()
```

```

    if i >= k - 1:
        out.append(a[dq[0]])
print(*out)

```

Đổi $\leq$ thành $\geq$ để tìm min.

Cốt lõi của ý tưởng

Khi a_i vào cửa sổ, mọi phần tử nhỏ hơn a_i và đứng trước đều vô dụng mãi mãi (vì a_i vừa lớn hơn vừa "sống lâu" hơn trong cửa sổ). Vậy ta xóa chúng.

5. Histogram & Maximum Rectangle (mẫu khó hơn)

Đề: Cho histogram với cột cao $h_1, \dots, h_n$ (rộng 1). Tìm hình chữ nhật có diện tích lớn nhất nằm gọn trong histogram.

Ý tưởng: Với mỗi cột i , tìm L_i = vị trí cột thấp hơn gần nhất bên trái, R_i = bên phải. Khi đó, hình chữ nhật cao h_i rộng nhất là từ $L_i + 1$ tới $R_i - 1$, diện tích $= h_i \times (R_i - L_i - 1)$.

Cả L và R đều tính bằng monotonic stack trong $O(n)$ – đây là bài toán quan trọng đáng nhớ.

Cài đặt (gói trong một hàm)

Mẹo cài đặt gọn: thêm hai cột "ảo" cao -1 ở đầu và cuối. Khi đó mọi cột thực sẽ có thấp hơn ở cả hai phía - không cần xử lý biên đặc biệt.

C++

```

long long largestRectangleInHistogram(vector<int> h) {
    int n = h.size();
    h.push_back(-1); // Cột "ảo" thấp hơn mọi cột -> bước pop hết
    stack<int> st;
    long long ans = 0;
    for (int i = 0; i <= n; i++) {
        while (!st.empty() && h[st.top()] >= h[i]) {
            int j = st.top(); st.pop();
            int left = st.empty() ? -1 : st.top();
            ans = max(ans, (long long) h[j] * (i - left - 1));
        }
        st.push(i);
    }
    return ans;
}

```

Python

```

def largest_rectangle_in_histogram(h):
    h = h + [-1] # Cột "ảo" thấp hơn mọi cột
    st = []
    ans = 0
    for i, x in enumerate(h):

```

```

while st and h[st[-1]] >= x:
    j = st.pop()
    left = st[-1] if st else -1
    ans = max(ans, h[j] * (i - left - 1))
st.append(i)
return ans

```

Cách hoạt động: khi pop cột j , vì cột i "thấp hơn (hoặc bằng)", ta biết i chính là biên phải R_j và đỉnh stack mới là biên trái L_j . Hình chữ nhật cao $h[j]$ rộng từ $L_j + 1$ tới $R_j - 1$.

Mở rộng: Hình chữ nhật lớn nhất toàn 1 trong ma trận nhị phân

Đề: Cho ma trận $m \times n$ gồm 0 và 1. Tìm hình chữ nhật con lớn nhất chỉ chứa toàn 1.

Ý tưởng: quy về histogram trên từng dòng:

1. Với mỗi ô (i, j) , $h_i[j]$ là số số 1 liên tiếp đi lên từ ô (i, j) .
2. $h_i[j] = h_{i-1}[j] + 1$ nếu $a[i][j] = 1$.
3. $h_i[j] = 0$ nếu $a[i][j] = 0$.
4. Sau đó dòng i trở thành một histogram với các cột cao $h_i[1], h_i[2], \dots, h_i[n]$. Hình chữ nhật toàn 1 lớn nhất kết thúc ở dòng i chính là max rectangle trong histogram đó.
5. Đáp án = max qua mọi dòng.

Phức tạp: $O(mn)$ – mỗi dòng dùng histogram max rect trong $O(n)$.

C++

```

int h[N] = {0}, ans = 0;
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        h[j] = (a[i][j] == 1) ? h[j] + 1 : 0;
    }
    ans = max(ans, (int)largestRectangleInHistogram(vector<int>(h + 1,
        h + n + 1)));
}

```

Python

```

h = [0] * (n + 1)
ans = 0
for i in range(1, m + 1):
    for j in range(1, n + 1):
        h[j] = h[j] + 1 if a[i][j] == 1 else 0
    ans = max(ans, largest_rectangle_in_histogram(h[1:]))

```

Đây là kỹ thuật giảm bài toán 2D về 1D – một mẫu rất phổ biến trong CP. Hãy nhớ kỹ!

6. Tổng kết & Mẹo

- Stack giải bài "ngoặc", "histogram", "nearest smaller/greater".
- Queue dùng cho BFS (sẽ thấy trong bài 9).
- Monotonic Deque giải sliding window min/max trong $O(n)$.
- Thao tác cốt lõi: "phần tử mới đến $\rightarrow$ xóa hết phần tử ở đuôi mà 'vô dụng' $\rightarrow$ push".
- Khi gặp "max/min trong mọi cửa sổ kích thước k ", hãy nghĩ monotonic deque.

Luyện tập là chìa khóa – làm 5-7 bài bạn sẽ nhận ra pattern ngay!